SOUGATA SAHA
IT-A2-037
00S ASSIGNMENT 1

ASSIGNEMT 1

Q1.Create a class “Room” which will hold the “height”, “width” and “breadth” of the room in three fields. This class also has a method “volume()” to calculate the volume of this room. Create another class “RoomDemo” which will use the earlier class, create instances of rooms, and display the volume of room.

Ans:

```
class room{
intheight,width,breadth;
room(inth,intw,int b ){
height=h;
width=w;
breadth=b;
   }
int volume(){
return height*width*breadth;
   }

}
classRoomdemo{
public static void main(String args[]){
roomob=new room(1,2,3);
int volume=ob.volume();
System.out.println("volume="+volume);
   }
}
```

Output:

```
"qu1.java" 19L, 380C written
[be2337@localhost Assignment1]$ javac qu1.java
[be2337@localhost Assignment1]$ java Roomdemo
volume=6
[be2337@localhost Assignment1]$
```

Q2.Write a program to implement a class “student” with the following members.

Name of the student.

Marks of the student obtained in three subjects.

Function to assign initial values.

Function to compute total average.

Function to display the student’s name and the total marks.

Write an appropriate main() function to demonstrate the functioning of the above.

Ans:

```
class student{
   String Name;
int m1,m2,m3;
student(String n){
      Name=n;
   }
voidset_marks(intm,intn,int o){
      m1=m;
      m2=n;
      m3=o;
   }
intget_avarage(){
return (m1+m2+m3)/3;
   }
void display(){
int total=m1+m2+m3;
System.out.println("Name:"+Name+" Total marks ="+total);
```

```
    }
public static void main (String[] args) {
student s=new student("SohomChakrabarty");
s.set_marks(90,90,90);
s.display();
    }
}
```

Output:

```
"qu2.java" 24L, 529C written
[be2337@localhost Assignment1]$ javac qu2.java
[be2337@localhost Assignment1]$ java student
Name:Sohom Chakrabarty Total marks =270
```

Qu3.Implement a class for stack of integers using an array. Please note that the operations defined for a stack data structure are as follows: “push”, “pop”, “print”. There should be a constructor to create an array of integers; the size of the array is provided by the user.

Write a main() function to (i) create a stack to hold maximum of 30 integers; (ii) push the numbers 10, 20, 30, 15, 9 to the stack; (iii) print the stack; (iii) pop thrice and (iv) print the stack again.

Ans:

```
class stack{
int top;
int size;
intarr[];
stack(int n){
top=-1;
arr=new int[n];
size=n;
    }
void push(int x){
if(top==size-1) {
System.out.println("stack is full ");
```

```
return;
        }
top++;
arr[top]=x;
    }
int pop(){
int x=-1;
if(top==-1){
System.out.println("Stack is empty");
return x;
        }
         x=arr[top];
top--;
return x;
    }

void show(){
for(int i=top;i>=0;i--){
System.out.println(arr[i]+" ");
        }
    }
public static void main (String[] args) {
stackst=new stack(30) ;
st.push(10);
st.push(20);
st.push(30);
st.push(15);
st.push(9);
st.show();
System.out.println(st.pop());
```

```
System.out.println(st.pop());
System.out.println(st.pop());
st.show();


    }
}
```

Output:

```
[be2337@localhost Assignment1]$ javac qu3.java
[be2337@localhost Assignment1]$ java stack
9
15
30
20
10
9
15
30
20
10
[be2337@localhost Assignment1]$
```

Q4.

```
classBankAccount {
intAccountNo;
    String ownername;
double balance;


BankAccount(intAccNo, String owner) {
ownername = owner;
AccountNo = AccNo;
balance = 0;
    }


void add(double amount) {
```

```
balance += amount;
    }

void subtract(double amount) {
if (amount > balance)
System.out.println("Insufficient Balance");
else
balance -= amount;
    }

voidget_detail() {
System.out.println("Owner Name: " + ownername);
System.out.println("AccountNo: " + AccountNo);
System.out.println("Balance: " + balance);
    }
}
classAccountManager {
BankAccount[] a;
intcurrAccount_no;

AccountManager() {
        a = new BankAccount[100];
currAccount_no = 1;
    }

void create(String name) {
a[currAccount_no] = new BankAccount(currAccount_no, name);
currAccount_no++;
    }
```

```
void deposit(intAccountno, double amount) {
a[Accountno].add(amount);
   }

void withdraw(intAccountno, double amount) {
a[Accountno].subtract(amount);
   }

voidprint_detail(intAccountno) {
if(a[Accountno]==null){
System.out.println("Account no do not exist");
return ;
  }
a[Accountno].get_detail();
   }
voiddelet(intAccountno){
a[Accountno]=null;
       }

}
class Bank {
public static void main(String args[]) {
AccountManager manager = new AccountManager();
manager.create("sougataSaha");
manager.create("sohomchakrabarty");
manager.print_detail(1);
manager.print_detail(2);
manager.delet(1);
manager.print_detail(1);
   }
```

}

```
[be2337@localhost Assignment1]$ javac qu4.java
[be2337@localhost Assignment1]$ java Bank
Owner Name: sougata Saha
AccountNo: 1
Balance: 0.0
Owner Name: sohom chakrabarty
AccountNo: 2
Balance: 0.0
Account no do not exist
[be2337@localhost Assignment1]$
```

Qu5.5. Write a class to represent complex numbers with necessary constructors. Write member functions to add, multiply two complex numbers.

There should be three constructors: (i) to initialize real and imaginary parts to 0; (ii) to initialize imaginary part to 0 but to initialize the real part to user defined value; (iii) to initialize the real part and the imaginary part to user defined values.

Also, write a main() function to (i) create two complex numbers 3+2i and 4-2i; (ii) to print the sum and product of those numbers.

```
class Complex {
double real, imaginary;

Complex() {
real = 0;
imaginary = 0;
   }

Complex(double r) {
real = r;
imaginary = 0;
   }
```

```
Complex(double r, double i) {
real = r;
imaginary = i;
    }

    Complex add(Complex c) {
return new Complex(real + c.real, imaginary + c.imaginary);
    }

    Complex multiply(Complex c) {
return new Complex(real * c.real - imaginary * c.imaginary, real * c.imaginary + imaginary * c.real);
    }

void display() {
System.out.println(real + " + " + imaginary + "i");
    }
public static void main(String[] args) {
        Complex c1 = new Complex(3, 2);
        Complex c2 = new Complex(4, -2);
        Complex sum = c1.add(c2);
        Complex product = c1.multiply(c2);
System.out.print("Sum: ");
sum.display();
System.out.print("Product: ");
product.display();
    }
}
```

Output:

```
[be2337@localhost Assignment1]$ javac qu5.java
[be2337@localhost Assignment1]$ java Complex
Sum: 7.0 + 0.0i
Product: 16.0 + 2.0i
```

Q6.Write a Java class “Employee” containing information name, id, address, salary etc. Write necessary constructor and read/write methods.

Ans:

```
class Employee {
    private String name;
    private int id;
    private String address;
    private double salary;

    public Employee(String name, int id, String address, double salary) {
        this.name = name;
        this.id = id;
        this.address = address;
        this.salary = salary;
    }

    public double getSalary() {
        return salary;
    }

    public void display() {
        System.out.println("Employee ID: " + id + ", Name: " + name + ", Address: " + address + ", Salary: " + salary);
    }
```

```
}

class Dept {
    String name;
    private String location;
    private Employee emp1, emp2, emp3, emp4, emp5;

    public Dept(String name, String location) {
        this.name = name;
        this.location = location;
    }

    public void add(Employee employee, int position) {
        switch (position) {
            case 1: emp1 = employee; break;
            case 2: emp2 = employee; break;
            case 3: emp3 = employee; break;
            case 4: emp4 = employee; break;
            case 5: emp5 = employee; break;
        }
    }

    public double getYearlyExpenditure() {
        double totalExpenditure = 0;
        if (emp1 != null) totalExpenditure += emp1.getSalary() * 12;
        if (emp2 != null) totalExpenditure += emp2.getSalary() * 12;
        if (emp3 != null) totalExpenditure += emp3.getSalary() * 12;
```

```
        if (emp4 != null) totalExpenditure += emp4.getSalary() * 12;
        if (emp5 != null) totalExpenditure += emp5.getSalary() * 12;
        return totalExpenditure;
    }

    public void displayEmployees() {
        System.out.println("Employees in the " + name + " Department:");
        if (emp1 != null) emp1.display();
        if (emp2 != null) emp2.display();
        if (emp3 != null) emp3.display();
        if (emp4 != null) emp4.display();
        if (emp5 != null) emp5.display();
    }
}

class demo {
    public static void main(String[] args) {
        Dept itDept = new Dept("Information Technology", "New York");

        Employee emp1 = new Employee("Alice", 101, "123 Street, NY", 5000);
        Employee emp2 = new Employee("Bob", 102, "456 Avenue, NY", 5500);
        Employee emp3 = new Employee("Charlie", 103, "789 Boulevard, NY", 6000);
        Employee emp4 = new Employee("David", 104, "321 Lane, NY", 4500);
        Employee emp5 = new Employee("Eve", 105, "654 Road, NY", 4800);

        itDept.add(emp1, 1);
        itDept.add(emp2, 2);
```

```
        itDept.add(emp3, 3);
        itDept.add(emp4, 4);
        itDept.add(emp5, 5);

        itDept.displayEmployees();

        double yearlyExpenditure = itDept.getYearlyExpenditure();
        System.out.println("Yearly Expenditure for " + itDept.name + " Department: " +
yearlyExpenditure);
```

Output:

```
            case 1: empl = employee; break;
"qu6.java" 85L, 2739C written
[be2337@localhost Assignment1]$ javac qu6.java
[be2337@localhost Assignment1]$ java demo
Employees in the Information Technology Department:
Employee ID: 101, Name: Alice, Address: 123 Street, NY, Salary: 5000.0
Employee ID: 102, Name: Bob, Address: 456 Avenue, NY, Salary: 5500.0
Employee ID: 103, Name: Charlie, Address: 789 Boulevard, NY, Salary: 6000.0
Employee ID: 104, Name: David, Address: 321 Lane, NY, Salary: 4500.0
Employee ID: 105, Name: Eve, Address: 654 Road, NY, Salary: 4800.0
Yearly Expenditure for Information Technology Department: 309600.0
```

Q7. Create an abstract class “Publication” with data members ‘noOfPages’, ‘price’, ‘publisherName’ etc. with their accessor/modifier functions. Now create two sub-classes “Book” and “Journal”. Create a class Library that contains a list of Publications. Write a main() function and create three Books and two Journals, add them to library and print the details of all publications.

Ans:

Ans:

```
abstract class Publication {
    int noOfPages;
    double price;
    String publisherName;

    public Publication(int noOfPages, double price, String publisherName) {
        this.noOfPages = noOfPages;
```

```
        this.price = price;
        this.publisherName = publisherName;
    }

    public int getNoOfPages() {
        return noOfPages;
    }

    public void setNoOfPages(int noOfPages) {
        this.noOfPages = noOfPages;
    }

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public String getPublisherName() {
        return publisherName;
    }

    public void setPublisherName(String publisherName) {
        this.publisherName = publisherName;
    }

    abstract void printDetails();
}
```

```
class Book extends Publication {
    String authorName;

    public Book(int noOfPages, double price, String publisherName, String authorName) {
        super(noOfPages, price, publisherName);
        this.authorName = authorName;
    }

    public String getAuthorName() {
        return authorName;
    }

    public void setAuthorName(String authorName) {
        this.authorName = authorName;
    }

    void printDetails() {
        System.out.println("Book - Author: " + authorName + ", Pages: " + noOfPages + ", Price: " + price + ",
Publisher: " + publisherName);
    }
}

class Journal extends Publication {
    String edition;

    public Journal(int noOfPages, double price, String publisherName, String edition) {
        super(noOfPages, price, publisherName);
        this.edition = edition;
    }
```

```
    public String getEdition() {
        return edition;
    }

    public void setEdition(String edition) {
        this.edition = edition;
    }

    void printDetails() {
        System.out.println("Journal - Edition: " + edition + ", Pages: " + noOfPages + ", Price: " + price + ",
Publisher: " + publisherName);
    }
}

class Library {
    static int no_of_pulication = 0;
    Publication arr[] = new Publication[100];

    public void addPublication(Publication publication) {
        arr[no_of_pulication] = publication;
        no_of_pulication++;
    }

    public void printLibraryDetails() {
        for (int i = 0; i < no_of_pulication; i++) {
            arr[i].printDetails();
        }
    }
}
```

```
class demo {
    public static void main(String[] args) {
        Library library = new Library();

        Book book1 = new Book(300, 25.5, "Pearson", "John Smith");
        Book book2 = new Book(400, 30.0, "O'Reilly", "Jane Doe");
        Book book3 = new Book(200, 15.0, "McGraw Hill", "Mark Lee");

        Journal journal1 = new Journal(50, 10.0, "Elsevier", "Volume 1");
        Journal journal2 = new Journal(60, 12.0, "Springer", "Volume 2");

        library.addPublication(book1);
        library.addPublication(book2);
        library.addPublication(book3);
        library.addPublication(journal1);
        library.addPublication(journal2);

        library.printLibraryDetails();
    }
}
```

```
[be2337@localhost Assignment1]$ javac qu7.java
[be2337@localhost Assignment1]$ java demo
Book - Author: John Smith, Pages: 300, Price: 25.5, Publisher: Pearson
Book - Author: Jane Doe, Pages: 400, Price: 30.0, Publisher: O'Reilly
Book - Author: Mark Lee, Pages: 200, Price: 15.0, Publisher: McGraw Hill
Journal - Edition: Volume 1, Pages: 50, Price: 10.0, Publisher: Elsevier
Journal - Edition: Volume 2, Pages: 60, Price: 12.0, Publisher: Springer
[be2337@localhost Assignment1]$
```

Q8. Implement a class for a “Person”. Person has data members ‘age’, ’weight’, ‘height’, ‘dateOfBirth’, ‘address’ with proper reader/write methods etc. Now create two subclasses “Employee” and “Student”. Employee will have additional data member ‘salary’, ‘dateOfJoining’, ‘experience’ etc. Student has data members ‘roll’, ‘listOfSubjects’, their marks and methods ‘calculateGrade’. Again create two sub-classes “Technician” and “Professor” from Employee. Professor has data members ‘courses’, ‘listOfAdvisee’ and their add/remove methods. Write a main() function to demonstrate the creation of objects of different classes and their method calls.

Ans:

```
class Account {
    int accountNumber;
    String holderName;
    double balance;

    public Account(int accountNumber, String holderName, double balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    public int getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(int accountNumber) {
        this.accountNumber = accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }
```

```
    public void setHolderName(String holderName) {
        this.holderName = holderName;
    }

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    void printDetails() {
        System.out.println("Account Number: " + accountNumber + ", Holder Name: " + holderName + ",
Balance: " + balance);
    }
}

class SavingsAccount extends Account {
    double interestRate;

    public SavingsAccount(int accountNumber, String holderName, double balance, double interestRate) {
        super(accountNumber, holderName, balance);
        this.interestRate = interestRate;
    }

    public double getInterestRate() {
        return interestRate;
    }
```

```
    public void setInterestRate(double interestRate) {
        this.interestRate = interestRate;
    }

    public double calculateYearlyInterest() {
        return (balance * interestRate) / 100;
    }

    @Override
    void printDetails() {
        super.printDetails();
        System.out.println("Interest Rate: " + interestRate + "%, Yearly Interest: " + calculateYearlyInterest());
    }
}

class CurrentAccount extends Account {

    public CurrentAccount(int accountNumber, String holderName, double balance) {
        super(accountNumber, holderName, balance);
    }

    @Override
    void printDetails() {
        super.printDetails();
    }
}

class Manager {
        static int no_of_Account=0;
        Account arr[]=new Account[100];
```

```
    public void addAccount(Account account) {
        arr[no_of_Account]=account;
        no_of_Account++;
    }

    public void printAllAccountDetails() {
        for (int i=0;i<no_of_Account;i++) {
            arr[i].printDetails();
        }
    }
}

class demo {
    public static void main(String[] args) {
        Manager manager = new Manager();

        SavingsAccount savingsAccount1 = new SavingsAccount(101, "Alice", 5000.0, 4.5);
        SavingsAccount savingsAccount2 = new SavingsAccount(102, "Bob", 8000.0, 3.2);

        CurrentAccount currentAccount1 = new CurrentAccount(201, "Charlie", 12000.0);
        CurrentAccount currentAccount2 = new CurrentAccount(202, "David", 15000.0);
        CurrentAccount currentAccount3 = new CurrentAccount(203, "Eve", 9000.0);

        manager.addAccount(savingsAccount1);
        manager.addAccount(savingsAccount2);
        manager.addAccount(currentAccount1);
        manager.addAccount(currentAccount2);
        manager.addAccount(currentAccount3);

        manager.printAllAccountDetails();
```

```
    }
}
```

Output:

```
[be2337@localhost Assignment1]$ javac qu8.java
[be2337@localhost Assignment1]$ java demo
Account Number: 101, Holder Name: Alice, Balance: 5000.0
Interest Rate: 4.5%, Yearly Interest: 225.0
Account Number: 102, Holder Name: Bob, Balance: 8000.0
Interest Rate: 3.2%, Yearly Interest: 256.0
Account Number: 201, Holder Name: Charlie, Balance: 12000.0
Account Number: 202, Holder Name: David, Balance: 15000.0
Account Number: 203, Holder Name: Eve, Balance: 9000.0
[be2337@localhost Assignment1]$
```

Q9.

```
import java.util.ArrayList;
import java.util.List;

class Person {
    private int age;
    private double weight;
    private double height;
    private String dateOfBirth;
    private String address;

    public Person(int age, double weight, double height, String dateOfBirth, String address) {
        this.age = age;
        this.weight = weight;
        this.height = height;
```

```
    this.dateOfBirth = dateOfBirth;
    this.address = address;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public double getWeight() {
    return weight;
}

public void setWeight(double weight) {
    this.weight = weight;
}

public double getHeight() {
    return height;
}

public void setHeight(double height) {
    this.height = height;
}
```

```
    public String getDateOfBirth() {
        return dateOfBirth;
    }

    public void setDateOfBirth(String dateOfBirth) {
        this.dateOfBirth = dateOfBirth;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public void printDetails() {
        System.out.println("Age: " + age);
        System.out.println("Weight: " + weight);
        System.out.println("Height: " + height);
        System.out.println("Date of Birth: " + dateOfBirth);
        System.out.println("Address: " + address);
    }
}

class Employee extends Person {
```

```
    private double salary;
    private String dateOfJoining;
    private int experience;

    public Employee(int age, double weight, double height, String dateOfBirth, String address,
double salary, String dateOfJoining, int experience) {
        super(age, weight, height, dateOfBirth, address);
        this.salary = salary;
        this.dateOfJoining = dateOfJoining;
        this.experience = experience;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public String getDateOfJoining() {
        return dateOfJoining;
    }

    public void setDateOfJoining(String dateOfJoining) {
        this.dateOfJoining = dateOfJoining;
    }
```

```
    public int getExperience() {
        return experience;
    }

    public void setExperience(int experience) {
        this.experience = experience;
    }

    @Override
    public void printDetails() {
        super.printDetails();
        System.out.println("Salary: " + salary);
        System.out.println("Date of Joining: " + dateOfJoining);
        System.out.println("Experience: " + experience + " years");
    }
}

class Student extends Person {
    private String roll;
    private List<String> listOfSubjects;
    private List<Double> marks;

    public Student(int age, double weight, double height, String dateOfBirth, String address,
String roll) {
        super(age, weight, height, dateOfBirth, address);
        this.roll = roll;
```

```
    this.listOfSubjects = new ArrayList<>();
    this.marks = new ArrayList<>();
}

public void addSubject(String subject, double mark) {
    listOfSubjects.add(subject);
    marks.add(mark);
}

public String calculateGrade() {
    double totalMarks = 0;
    for (Double mark : marks) {
        totalMarks += mark;
    }
    double average = totalMarks / marks.size();
    if (average >= 90) {
        return "A";
    } else if (average >= 75) {
        return "B";
    } else if (average >= 50) {
        return "C";
    } else {
        return "D";
    }
}

@Override
```

```
    public void printDetails() {
        super.printDetails();
        System.out.println("Roll: " + roll);
        System.out.println("Subjects: " + listOfSubjects);
        System.out.println("Grade: " + calculateGrade());
    }
}

class Technician extends Employee {
    public Technician(int age, double weight, double height, String dateOfBirth, String address,
double salary, String dateOfJoining, int experience) {
        super(age, weight, height, dateOfBirth, address, salary, dateOfJoining, experience);
    }

    @Override
    public void printDetails() {
        System.out.println("Technician Details:");
        super.printDetails();
    }
}

class Professor extends Employee {
    private List<String> courses;
    private List<String> listOfAdvisee;

    public Professor(int age, double weight, double height, String dateOfBirth, String address,
double salary, String dateOfJoining, int experience) {
        super(age, weight, height, dateOfBirth, address, salary, dateOfJoining, experience);
```

```
    this.courses = new ArrayList<>();
    this.listOfAdvisee = new ArrayList<>();
}

public void addCourse(String course) {
    courses.add(course);
}

public void removeCourse(String course) {
    courses.remove(course);
}

public void addAdvisee(String advisee) {
    listOfAdvisee.add(advisee);
}

public void removeAdvisee(String advisee) {
    listOfAdvisee.remove(advisee);
}

@Override
public void printDetails() {
    System.out.println("Professor Details:");
    super.printDetails();
    System.out.println("Courses: " + courses);
    System.out.println("Advisees: " + listOfAdvisee);
}
```

```
}


 class demo {
    public static void main(String[] args) {
        Technician technician = new Technician(30, 70, 5.8, "1994-05-10", "123 Tech Street",
50000, "2020-01-10", 5);
        Professor professor = new Professor(45, 80, 6.0, "1979-03-15", "456 University Ave",
80000, "2010-08-15", 15);


        professor.addCourse("Data Structures");
        professor.addCourse("Algorithms");
        professor.addAdvisee("Alice");
        professor.addAdvisee("Bob");


        Student student = new Student(20, 60, 5.6, "2004-07-25", "789 College Road", "S12345");
        student.addSubject("Math", 88);
        student.addSubject("English", 92);


        technician.printDetails();
        System.out.println();
        professor.printDetails();
        System.out.println();
        student.printDetails();
    }
}
```

Output:

```
"qu9.java" 224L, 6186C written
[be2337@localhost Assignment1]$ javac qu9.java
[be2337@localhost Assignment1]$ java demo
Technician Details:
Age: 30
Weight: 70.0
Height: 5.8
Date of Birth: 1994-05-10
Address: 123 Tech Street
Salary: 50000.0
Date of Joining: 2020-01-10
Experience: 5 years

Professor Details:
Age: 45
Weight: 80.0
Height: 6.0
Date of Birth: 1979-03-15
Address: 456 University Ave
Salary: 80000.0
Date of Joining: 2010-08-15
Experience: 15 years
Courses: [Data Structures, Algorithms]
Advisees: [Alice, Bob]

Age: 20
Weight: 60.0
Height: 5.6
Date of Birth: 2004-07-25
Address: 789 College Road
Roll: S12345
Subjects: [Math, English]
Grade: A
```

Q10. . class Book {

String author;

String title;

String publisher;

double cost;

int stock;

public Book(String author, String title, String publisher, double cost, int stock) {

```
        this.author = author;
        this.title = title;
        this.publisher = publisher;
        this.cost = cost;
        this.stock = stock;
    }

    public boolean isAvailable(String searchTitle, String searchAuthor) {
        return this.title.equals(searchTitle) && this.author.equals(searchAuthor);
    }

    public void displayDetails() {
        System.out.println("Title: " + title + ", Author: " + author + ", Publisher: " + publisher + ", Cost: " + cost
+ ", Stock: " + stock);
    }

    public boolean canFulfillOrder(int quantity) {
        return stock >= quantity;
    }

    public void reduceStock(int quantity) {
        stock -= quantity;
    }

    public double calculateTotalCost(int quantity) {
        return cost * quantity;
    }
}

class BookShop {
```

```
    Book[] books;

    public BookShop(Book[] books) {
        this.books = books;
    }

    public void searchAndOrderBook(String title, String author, int quantity) {
        for (Book book : books) {
            if (book.isAvailable(title, author)) {
                book.displayDetails();
                if (book.canFulfillOrder(quantity)) {
                    double totalCost = book.calculateTotalCost(quantity);
                    System.out.println("Total Cost: " + totalCost);
                    book.reduceStock(quantity);
                } else {
                    System.out.println("Requires copies not in stock.");
                }
                return;
            }
        }
        System.out.println("Book not available.");
    }
}

public class Main {
    public static void main(String[] args) {
        Book book1 = new Book("J.K. Rowling", "Harry Potter", "Bloomsbury", 15.99, 10);
        Book book2 = new Book("J.R.R. Tolkien", "The Hobbit", "HarperCollins", 12.99, 5);
        Book book3 = new Book("George Orwell", "1984", "Secker & Warburg", 9.99, 0);
```

```
        Book[] books = {book1, book2, book3};
        BookShop shop = new BookShop(books);

        shop.searchAndOrderBook("Harry Potter", "J.K. Rowling", 3);
        shop.searchAndOrderBook("The Hobbit", "J.R.R. Tolkien", 7);
        shop.searchAndOrderBook("1984", "George Orwell", 1);
    }
}
```

```
qu10.java" 67L, 2238C written
[be2337@localhost Assignment1]$ javac qu10.java
[be2337@localhost Assignment1]$ java Bookshop
Book is available!
Title: The Alchemist
Author: Paulo Coelho
Publisher: HarperOne
Cost: $15.99
Stock: 10 copies
Total cost: $47.97
[be2337@localhost Assignment1]$
```

Q11.

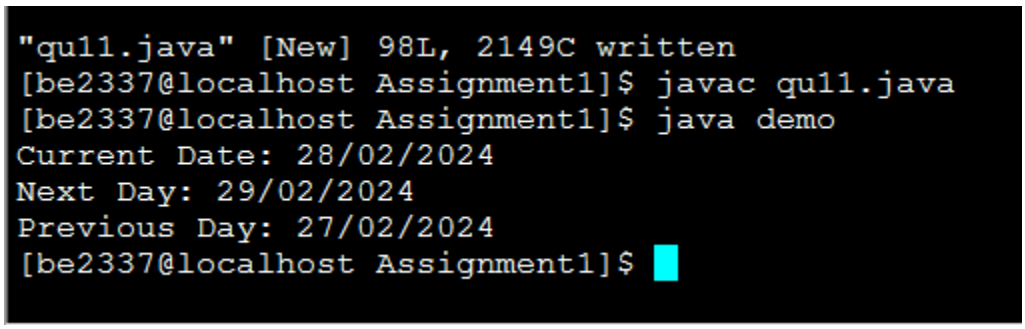
Ans:

```
class Date {
    private int day;
    private int month;
    private int year;

    public Date() {
        this.day = 1;
        this.month = 1;
        this.year = 1970;
    }
```

```
public Date(int day) {
    this.day = day;
    this.month = 1;
    this.year = 1970;
}

public Date(int day, int month) {
    this.day = day;
    this.month = month;
    this.year = 1970;
}

public Date(int day, int month, int year) {
    this.day = day;
    this.month = month;
    this.year = year;
}

public void printDate() {
    System.out.printf("%02d/%02d/%d%n", day, month, year);
}

public void getNextDay() {
    if (isLastDayOfMonth()) {
        if (month == 12) {
            day = 1;
            month = 1;
            year++;
        } else {
```

```
            day = 1;
            month++;
        }
    } else {
        day++;
    }
}

public void getPreviousDay() {
    if (day == 1) {
        if (month == 1) {
            month = 12;
            year--;
            day = daysInMonth();
        } else {
            month--;
            day = daysInMonth();
        }
    } else {
        day--;
    }
}

private boolean isLeapYear() {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}

private boolean isLastDayOfMonth() {
    return day == daysInMonth();
}
```

```
    private int daysInMonth() {
        switch (month) {
            case 2:
                return isLeapYear() ? 29 : 28;
            case 4: case 6: case 9: case 11:
                return 30;
            default:
                return 31;
        }
    }
}
class demo {
    public static void main(String[] args) {
        Date date = new Date(29, 1, 2025);

        System.out.print("Current Date: ");
        date.printDate();

        date.getNextDay();
        System.out.print("Next Day: ");
        date.printDate();

        date.getPreviousDay();
        date.getPreviousDay();
        System.out.print("Previous Day: ");
        date.printDate();
    }
}
```



Q12. Implement a class for a “Student”. Information about a student includes name, roll no and an array of five subject names. The class should have suitable constructor and get/set methods.

Implement a class “TabulationSheet”. A tabulation sheet contains roll numbers and marks of each student for a particular subject. This class should have a method for adding the marks and roll no of a student.

Implement a class “MarkSheet”. A mark sheet contains marks of all subjects for a particular student. This class should have a method to add name of a student and marks in each subject.

Write a main() function to create three “Student” objects, Five “Tabulationsheet” objects for Five subjects and three “Marksheet” object for three students. Print the mark sheets.

Ans:

```
import java.util.HashMap;


class Student {
    private String name;
    private int rollNo;
    private String[] subjects;

    public Student(String name, int rollNo, String[] subjects) {
        this.name = name;
        this.rollNo = rollNo;
        this.subjects = subjects;
    }
```

```
    public String getName() {
        return name;
    }

    public int getRollNo() {
        return rollNo;
    }

    public String[] getSubjects() {
        return subjects;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }

    public void setSubjects(String[] subjects) {
        this.subjects = subjects;
    }
}

class TabulationSheet {
```

```
    private String subject;
    private HashMap<Integer, Integer> marksByRollNo;

    public TabulationSheet(String subject) {
        this.subject = subject;
        this.marksByRollNo = new HashMap<>();
    }

    public void addMarks(int rollNo, int marks) {
        marksByRollNo.put(rollNo, marks);
    }

    public int getMarks(int rollNo) {
        return marksByRollNo.getOrDefault(rollNo, 0);
    }

    public String getSubject() {
        return subject;
    }
}

class MarkSheet {
    private String studentName;
    private int rollNo;
    private HashMap<String, Integer> marksBySubject;

    public MarkSheet(String studentName, int rollNo) {
```

```
        this.studentName = studentName;
        this.rollNo = rollNo;
        this.marksBySubject = new HashMap<>();
    }

    public void addMarks(String subject, int marks) {
        marksBySubject.put(subject, marks);
    }

    public void printMarkSheet() {
        System.out.println("Mark Sheet for: " + studentName);
        System.out.println("Roll No: " + rollNo);
        for (String subject : marksBySubject.keySet()) {
            System.out.println(subject + ": " + marksBySubject.get(subject));
        }
        System.out.println();
    }
}

public class Main {
    public static void main(String[] args) {
        String[] subjects = { "Math", "Physics", "Chemistry", "Biology", "English" };

        Student student1 = new Student("Alice", 101, subjects);
        Student student2 = new Student("Bob", 102, subjects);
        Student student3 = new Student("Charlie", 103, subjects);
```

```
TabulationSheet mathSheet = new TabulationSheet("Math");
TabulationSheet physicsSheet = new TabulationSheet("Physics");
TabulationSheet chemistrySheet = new TabulationSheet("Chemistry");
TabulationSheet biologySheet = new TabulationSheet("Biology");
TabulationSheet englishSheet = new TabulationSheet("English");

mathSheet.addMarks(101, 85);
mathSheet.addMarks(102, 90);
mathSheet.addMarks(103, 75);

physicsSheet.addMarks(101, 88);
physicsSheet.addMarks(102, 78);
physicsSheet.addMarks(103, 85);

chemistrySheet.addMarks(101, 92);
chemistrySheet.addMarks(102, 87);
chemistrySheet.addMarks(103, 80);

biologySheet.addMarks(101, 84);
biologySheet.addMarks(102, 79);
biologySheet.addMarks(103, 83);

englishSheet.addMarks(101, 90);
englishSheet.addMarks(102, 85);
englishSheet.addMarks(103, 88);

MarkSheet markSheet1 = new MarkSheet(student1.getName(), student1.getRollNo());
```

```
        MarkSheet markSheet2 = new MarkSheet(student2.getName(), student2.getRollNo());
        MarkSheet markSheet3 = new MarkSheet(student3.getName(), student3.getRollNo());

        markSheet1.addMarks("Math", mathSheet.getMarks(101));
        markSheet1.addMarks("Physics", physicsSheet.getMarks(101));
        markSheet1.addMarks("Chemistry", chemistrySheet.getMarks(101));
        markSheet1.addMarks("Biology", biologySheet.getMarks(101));
        markSheet1.addMarks("English", englishSheet.getMarks(101));

        markSheet2.addMarks("Math", mathSheet.getMarks(102));
        markSheet2.addMarks("Physics", physicsSheet.getMarks(102));
        markSheet2.addMarks("Chemistry", chemistrySheet.getMarks(102));
        markSheet2.addMarks("Biology", biologySheet.getMarks(102));
        markSheet2.addMarks("English", englishSheet.getMarks(102));

        markSheet3.addMarks("Math", mathSheet.getMarks(103));
        markSheet3.addMarks("Physics", physicsSheet.getMarks(103));
        markSheet3.addMarks("Chemistry", chemistrySheet.getMarks(103));
        markSheet3.addMarks("Biology", biologySheet.getMarks(103));
        markSheet3.addMarks("English", englishSheet.getMarks(103));

        markSheet1.printMarkSheet();
        markSheet2.printMarkSheet();
        markSheet3.printMarkSheet();
    }
}
```

```
[be2337@localhost Assignment1]$ javac qu12.java
[be2337@localhost Assignment1]$ java demo
Mark Sheet for: Alice
Roll No: 101
English: 90
Chemistry: 92
Biology: 84
Math: 85
Physics: 88

Mark Sheet for: Bob
Roll No: 102
English: 85
Chemistry: 87
Biology: 79
Math: 90
Physics: 78

Mark Sheet for: Charlie
Roll No: 103
English: 88
Chemistry: 80
Biology: 83
Math: 75
Physics: 85
```

Q13. Create a base class “Automobile”. An Automobile contains data members ‘make’, ‘type’, ‘maxSpeed’, ‘price’, ‘mileage’, ‘registrationNumber’ etc. with their reader/writer methods. Now create two sub-classes “Track” and “Car”. Track has data members ‘capacity’, ‘hoodType’, ‘noOfWheels’ etc. Car has data members ‘noOfDoors’, ‘seatingCapacity’ and their reader/writer methods. Create a main() function to demonstrate this.

Ans:

```
class Automobile {
    private String make;
    private String type;
    private int maxSpeed;
    private double price;
    private double mileage;
```

```
    private String registrationNumber;

    public Automobile(String make, String type, int maxSpeed, double price, double mileage, String
registrationNumber) {
        this.make = make;
        this.type = type;
        this.maxSpeed = maxSpeed;
        this.price = price;
        this.mileage = mileage;
        this.registrationNumber = registrationNumber;
    }

    public String getMake() {
        return make;
    }

    public void setMake(String make) {
        this.make = make;
    }

    public String getType() {
        return type;
    }

    public void setType(String type) {
        this.type = type;
    }

    public int getMaxSpeed() {
        return maxSpeed;
```

```
}

public void setMaxSpeed(int maxSpeed) {
    this.maxSpeed = maxSpeed;
}

public double getPrice() {
    return price;
}

public void setPrice(double price) {
    this.price = price;
}

public double getMileage() {
    return mileage;
}

public void setMileage(double mileage) {
    this.mileage = mileage;
}

public String getRegistrationNumber() {
    return registrationNumber;
}

public void setRegistrationNumber(String registrationNumber) {
    this.registrationNumber = registrationNumber;
}
```

```
    public void displayDetails() {
        System.out.println("Make: " + make);
        System.out.println("Type: " + type);
        System.out.println("Max Speed: " + maxSpeed + " km/h");
        System.out.println("Price: $" + price);
        System.out.println("Mileage: " + mileage + " km/l");
        System.out.println("Registration Number: " + registrationNumber);
    }
}

class Track extends Automobile {
    private int capacity;
    private String hoodType;
    private int noOfWheels;

    public Track(String make, String type, int maxSpeed, double price, double mileage, String
registrationNumber, int capacity, String hoodType, int noOfWheels) {
        super(make, type, maxSpeed, price, mileage, registrationNumber);
        this.capacity = capacity;
        this.hoodType = hoodType;
        this.noOfWheels = noOfWheels;
    }

    public int getCapacity() {
        return capacity;
    }

    public void setCapacity(int capacity) {
        this.capacity = capacity;
    }
```

```
    public String getHoodType() {
        return hoodType;
    }

    public void setHoodType(String hoodType) {
        this.hoodType = hoodType;
    }

    public int getNoOfWheels() {
        return noOfWheels;
    }

    public void setNoOfWheels(int noOfWheels) {
        this.noOfWheels = noOfWheels;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Capacity: " + capacity + " tons");
        System.out.println("Hood Type: " + hoodType);
        System.out.println("Number of Wheels: " + noOfWheels);
    }
}

class Car extends Automobile {
    private int noOfDoors;
    private int seatingCapacity;
```

```
    public Car(String make, String type, int maxSpeed, double price, double mileage, String
registrationNumber, int noOfDoors, int seatingCapacity) {
        super(make, type, maxSpeed, price, mileage, registrationNumber);
        this.noOfDoors = noOfDoors;
        this.seatingCapacity = seatingCapacity;
    }

    public int getNoOfDoors() {
        return noOfDoors;
    }

    public void setNoOfDoors(int noOfDoors) {
        this.noOfDoors = noOfDoors;
    }

    public int getSeatingCapacity() {
        return seatingCapacity;
    }

    public void setSeatingCapacity(int seatingCapacity) {
        this.seatingCapacity = seatingCapacity;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Number of Doors: " + noOfDoors);
        System.out.println("Seating Capacity: " + seatingCapacity);
    }
}
```

```
class demo {
    public static void main(String[] args) {
        Track track = new Track("Volvo", "Truck", 120, 50000, 8, "TRK1234", 10, "Open", 6);
        Car car = new Car("Toyota", "Sedan", 180, 30000, 15, "CAR5678", 4, 5);

        System.out.println("Track Details:");
        track.displayDetails();

        System.out.println("\nCar Details:");
        car.displayDetails();
    }
}
```

```
"qu13.java" [New] 165L, 4363C written
[be2337@localhost Assignment1]$ javac qu13.java
[be2337@localhost Assignment1]$ java demo
Track Details:
Make: Volvo
Type: Truck
Max Speed: 120 km/h
Price: $50000.0
Mileage: 8.0 km/l
Registration Number: TRK1234
Capacity: 10 tons
Hood Type: Open
Number of Wheels: 6

Car Details:
Make: Toyota
Type: Sedan
Max Speed: 180 km/h
Price: $30000.0
Mileage: 15.0 km/l
Registration Number: CAR5678
Number of Doors: 4
Seating Capacity: 5
[be2337@localhost Assignment1]$
```

Q14. Implement the classes for the following inheritance hierarchies.

Create an interface “Shape” that contains methods ‘area’, ‘draw’, ‘rotate’, ‘move’ etc. Now create two classes “Circle” and “Rectangle” that implement this ‘Shape’ interface and implement the methods ‘area’, ‘move’, etc. Write a main() function to create two “Circle” and three “Rectangle”, then move them. Print the details of circles and rectangles before after moving them.

Ans:

```
interface Shape {
   double area();
   void draw();
   void rotate();
   void move();
}
```

```
class Circle implements Shape {
    double radius;
    double x, y;

    Circle(double radius, double x, double y) {
        this.radius = radius;
        this.x = x;
        this.y = y;
    }

    public double area() {
        return Math.PI * radius * radius;
    }

    public void draw() {
        System.out.println("Drawing Circle at (" + x + ", " + y + ")");
    }

    public void rotate() {
        System.out.println("Rotating Circle at (" + x + ", " + y + ")");
    }

    public void move() {
        x += 5;
        y += 5;
    }

    void print_data(){
        System.err.println("Circle: radius = " + radius + ", position = (" + x + ", " + y + ")");
    }
```

```
}

class Rectangle implements Shape {
    double width, height;
    double x, y;

    Rectangle(double width, double height, double x, double y) {
        this.width = width;
        this.height = height;
        this.x = x;
        this.y = y;
    }

    public double area() {
        return width * height;
    }

    public void draw() {
        System.out.println("Drawing Rectangle at (" + x + ", " + y + ")");
    }

    public void rotate() {
        System.out.println("Rotating Rectangle at (" + x + ", " + y + ")");
    }

    public void move() {
        x += 5;
        y += 5;
    }
```

```
    void print_data(){
        System.err.println("Rectangle: width = " + width + ", height = " + height + ", position = (" + x + ", " + y +
")");
    }
}
class demo {
    public static void main(String[] args) {
        Circle circle1 = new Circle(5, 0, 0);
        Circle circle2 = new Circle(3, 10, 10);
        Rectangle rect1 = new Rectangle(4, 6, 1, 1);
        Rectangle rect2 = new Rectangle(7, 8, 5, 5);
        Rectangle rect3 = new Rectangle(6, 9, 8, 8);

        System.out.println("Before Moving:");
        circle1.print_data();
        circle2.print_data();
        rect1.print_data();
        rect2.print_data();
        rect3.print_data();

        circle1.move();
        circle2.move();
        rect1.move();
        rect2.move();
        rect3.move();

        System.out.println("\nAfter Moving:");
        circle1.print_data();
        circle2.print_data();
```

```
        rect1.print_data();
        rect2.print_data();
        rect3.print_data();

    }
}
```

```
[be2337@localhost Assignment1]$ javac q14.java
[be2337@localhost Assignment1]$ java demo
Before Moving:
Circle: radius = 5.0, position = (0.0, 0.0)
Circle: radius = 3.0, position = (10.0, 10.0)
Rectangle: width = 4.0, height = 6.0, position = (1.0, 1.0)
Rectangle: width = 7.0, height = 8.0, position = (5.0, 5.0)
Rectangle: width = 6.0, height = 9.0, position = (8.0, 8.0)

After Moving:
Circle: radius = 5.0, position = (5.0, 5.0)
Circle: radius = 3.0, position = (15.0, 15.0)
Rectangle: width = 4.0, height = 6.0, position = (6.0, 6.0)
Rectangle: width = 7.0, height = 8.0, position = (10.0, 10.0)
Rectangle: width = 6.0, height = 9.0, position = (13.0, 13.0)
[be2337@localhost Assignment1]$
```

Q15. Imagine a toll booth and a bridge. Cars passing by the booth are expected to pay an amount of Rs. 50/- as toll tax. Mostly they do but sometimes a car goes by without paying. The toll booth keeps track of the number of the cars that have passed without paying, total number of cars passed by, and the total amount of money collected. Execute this with a class called “Tollbooth” and print out the result as follows:

The total number of cars passed by without paying.

Total number of cars passed by.

Total cash collected.

Ans:

:
```
class Tollbooth {
    private int totalCars;
    private int carsWithoutPaying;
    private int totalCash;

    public Tollbooth() {
        totalCars = 0;
```

```
        carsWithoutPaying = 0;
        totalCash = 0;
    }

    public void carPassed(boolean paid) {
        totalCars++;
        if (paid) {
            totalCash += 50;
        } else {
            carsWithoutPaying++;
        }
    }

    public void displayDetails() {
        System.out.println("The total number of cars passed by without paying: " + carsWithoutPaying);
        System.out.println("Total number of cars passed by: " + totalCars);
        System.out.println("Total cash collected: Rs. " + totalCash);
    }
}
 class demo {
    public static void main(String[] args) {
        Tollbooth tollbooth = new Tollbooth();
        tollbooth.carPassed(true);
        tollbooth.carPassed(false);
        tollbooth.carPassed(true);
        tollbooth.carPassed(false);
        tollbooth.displayDetails();
    }
}
```

```
[be2337@localhost Assignment1]$ javac qu15.java
[be2337@localhost Assignment1]$ java demo
The total number of cars passed by without paying: 2
Total number of cars passed by: 4
Total cash collected: Rs. 100
[be2337@localhost Assignment1]$
```

Q16. . Write two interfaces “Fruit” and “Vegetable” containing methods ‘hasAPeel’ and ‘hasARoot’.

Now write a class “Tomato” implementing Fruit and Vegetable. Instantiate an object of Tomato.

Print the details of this object.

Ans:

```
interface Fruit {
    void hasAPeel();
}

interface Vegetable {
    void hasARoot();
}

class Tomato implements Fruit, Vegetable {
    public void hasAPeel() {
        System.out.println("Tomato has a peel.");
    }

    public void hasARoot() {
        System.out.println("Tomato has a root.");
    }

    void tomoto_own(){
        System.err.println("Tomato is both fruit and vegetable");
    }
}
```

```
class demo {
   public static void main(String[] args) {
      Tomato tomato = new Tomato();
      tomato.tomoto_own();
      tomato.hasAPeel();
      tomato.hasARoot();
   }
}
```

```
[be2337@localhost Assignment1]$ javac qu16.java
[be2337@localhost Assignment1]$ java demo
Tomato is both a fruit and a vegetable.
Tomato has a peel.
Tomato has a root.
[be2337@localhost Assignment1]$
```